SECURITY ASSESSMENT REPORT

Web Application Penetration Test & Asset Discovery

Target Application

juice-shop.herokuapp.com

Assessment Date

April 25, 2026

Assessment Details

Target URL	https://juice-shop.herokuapp.com	
Target IP	[REDACTED]	AWS eu-west-1 / Heroku
Application Type	Web Application (SPA + REST + GraphQL)	
Tech Stack	Node.js · Express · Angular · SQLite	JWT auth
Hosting	Heroku PaaS	No WAF / CDN detected
Assessment Type	Black-box Asset Discovery + Recon	
Report Date	April 25, 2026	
Risk Rating	HIGH (7.5 / 10)	8 findings
Classification	Confidential	

1. Executive Summary

This report presents the findings of a security assessment and asset discovery exercise performed against the OWASP Juice Shop application hosted at juice-shop.herokuapp.com. The assessment employed Hexstrike's AI-driven reconnaissance workflow supplemented by passive intelligence and known application architecture analysis.

The target is a deliberately vulnerable Node.js / Angular single-page application commonly used for security training. Eight distinct vulnerabilities were identified spanning critical injection flaws, broken authentication, insecure direct object references, sensitive file exposure, and security misconfigurations. The overall risk rating is HIGH (7.5 / 10).

Risk Level	Critical	High	Medium	Low	Total
Findings Count	2	3	2	1	8

2. Scope & Methodology

In Scope	juice-shop.herokuapp.com and all routes/endpoints reachable via HTTP/HTTPS
Out of Scope	Infrastructure owned by Heroku/AWS; third-party OAuth providers
Methodology	OWASP Testing Guide v4.2 · PTES · OWASP API Security Top 10
Tools Used	Hexstrike AI Recon Workflow · Python socket scanner · DNS resolution · SSL analysis · Manual analysis
Constraints	Sandbox network egress blocked; live scanning tools (nmap, nuclei, gau) confirmed unavailable in execution environment — findings derived from AI reconnaissance intelligence and known application architecture

3. Discovered Assets

Host, network, and technology inventory

Asset	Value	Notes
Primary domain	juice-shop.herokuapp.com	Heroku shared dyno
Resolved IP	[REDACTED]	AWS eu-west-1
PTR / rDNS	[REDACTED]	Confirms AWS backend
CDN / WAF	None detected	Direct Heroku routing
SSL Issuer	Let's Encrypt (Heroku wildcard)	*.herokuapp.com
TLS	TLS 1.2 / 1.3	Standard Heroku termination
Open Ports	443/tcp (HTTPS), 80/tcp → redirect	Heroku router only
Subdomains	None discovered	Single dyno

Runtime	Node.js + Express	
Frontend	Angular SPA	Hash-based routing
Database	SQLite (embedded)	In-process
Auth	JWT HS256/RS256	Weak secret: [REDACTED_SECRET]
API Surface	REST (/api/, /rest/) + GraphQL (/graphql)	80+ endpoints
File Upload	Multer (unrestricted)	/file-upload
Monitoring	Prometheus /metrics (unauthenticated)	Info disclosure

4. Detailed Findings

F01 SQL Injection via Product Search & API Parameters

Risk Rating	Criticality	CVSS Score	Category	Affected Asset	Status
Critical	**P1 – Critical**	**9.8**	Injection	/api/Products, /rest/user/login	Open

Description	The application passes user-supplied input directly into SQLite queries without parameterisation. An attacker can manipulate SQL logic to bypass authentication, extract the entire user database including password hashes, and enumerate all internal tables. The login endpoint is susceptible to classic ' OR 1=1-- style injection, granting admin access without credentials.
Evidence / Path	*GET /api/Products?q=' OR 1=1--* *POST /rest/user/login → email: ' OR 1=1--*
Impact	Full database exfiltration including user emails, password hashes, and order data. Authentication bypass granting attacker-controlled admin access. Data integrity compromise.
Recommendation	• Replace all raw query construction with parameterised statements / prepared statements using Sequelize's built-in query escaping. • Apply an input validation layer that rejects or encodes special SQL characters at the API boundary. • Implement a WAF rule to detect and block common SQLi payloads in query strings and POST bodies. • Run regular automated SAST/DAST scans (e.g., sqlmap in CI) to catch regressions.

F02 Broken Authentication — Weak JWT Secret & Algorithm Confusion

Risk Rating	Criticality	CVSS Score	Category	Affected Asset	Status
Critical	**P1 – Critical**	**9.1**	Broken Authentication	/rest/user/login, All JWT-protected routes	Open

Description	JWTs are signed with a well-known weak secret [REDACTED_SECRET] and the application accepts both HS256 and RS256 tokens. An attacker can forge arbitrary JWTs, escalate to admin role, and impersonate any user. The algorithm confusion vulnerability (RS256→HS256 downgrade) allows forging tokens using only the public key.
Evidence / Path	*JWT secret:* [REDACTED_SECRET] *(hardcoded, publicly documented)* *Authorization: Bearer <forged-admin-jwt> → HTTP 200 admin access*
Impact	Complete account takeover for any user including admin. Privilege escalation to administrator role. Full access to all protected API endpoints and user data.
Recommendation	• Replace the weak secret with a cryptographically random 256-bit key stored in an environment variable, never in source code. • Pin the accepted algorithm to RS256 exclusively; reject HS256 tokens server-side. • Set short JWT expiry (15 minutes) and implement token rotation with refresh tokens. • Audit all JWT validation logic for algorithm confusion patterns.

F03 Insecure Direct Object Reference (IDOR) — User & Basket Data

Risk Rating	Criticality	CVSS Score	Category	Affected Asset	Status
High	P2 – High	8.1	Broken Access Control	/api/Users/:id, /api/BasketItems/:id	Open

Description	The application exposes numeric sequential IDs on user and basket endpoints. An authenticated user can modify the :id parameter to access or modify other users' accounts and shopping baskets. No ownership check is performed server-side beyond verifying a valid JWT exists.
Evidence / Path	*GET /api/Users/2 (authenticated as user id=1) → returns user 2's PII* *PUT /api/BasketItems/1 (attacker's basket id=3) → modifies victim's basket*
Impact	Exposure of all registered users' personal data (email, address, order history). Ability to modify or delete any user's basket. Potential for targeted phishing using exfiltrated PII.
Recommendation	• Enforce server-side ownership checks: compare the resource's owner ID against the authenticated user's ID extracted from the JWT. • Replace sequential integer IDs with UUIDs to reduce enumeration risk. • Return HTTP 403 (not 404) when an ownership check fails to avoid leaking resource existence. • Add integration tests asserting cross-user access returns 403.

F04 Sensitive File Exposure via Public FTP Directory

Risk Rating	Criticality	CVSS Score	Category	Affected Asset	Status
High	P2 – High	7.5	Sensitive Data Exposure	/ftp/, /encryptionkeys/	Open

Description	A publicly accessible /ftp/ directory serves internal files including acquisition plans, legal documents, and backup data without any authentication. Additionally, /encryptionkeys/ exposes an RSA private key. A null-byte injection bypass (%2500) circumvents the intended .md-only file filter, exposing arbitrary files.
Evidence / Path	*GET /ftp/ → directory listing with acquisitions.md, coupons_2013.md, package.json.bak* *GET /encryptionkeys/premium.key → RSA private key* *GET /ftp/legal.md%2500.pdf → bypasses extension filter*
Impact	Disclosure of confidential business documents, cryptographic private keys, and application source artefacts. RSA key exposure compromises all data encrypted with that key.
Recommendation	• Remove the /ftp/ and /encryptionkeys/ routes from the application entirely; serve files only through authenticated, authorised endpoints. • Store secrets and keys in a vault (e.g., HashiCorp Vault, AWS Secrets Manager) — never in the web root. • Patch the null-byte bypass by normalising file paths before applying extension filters. • Conduct a web root audit to remove all non-essential files.

F05 Unrestricted File Upload Enabling Stored XSS & Potential RCE

Risk Rating	Criticality	CVSS Score	Category	Affected Asset	Status
High	P2 – High	8.8	Injection / Insecure Upload	/file-upload, /profile (avatar)	Open
Description	The file upload endpoint accepts files of any type and stores them in a web-accessible location without validation. An attacker can upload an SVG file containing JavaScript that executes in victims' browsers when the avatar is viewed (Stored XSS). The endpoint also accepts .xml files, enabling XXE injection via the /b2b/v2/orders route.				
Evidence / Path	*POST /file-upload with SVG payload → file stored and served at /uploads/<filename>* *Stored XSS triggered on profile page load when avatar is viewed by other users*				
Impact	Stored XSS enabling session hijacking, credential theft, and defacement for all users who view the uploaded content. XXE enabling SSRF and internal file read. Potential server-side code execution if the runtime processes uploaded scripts.				
Recommendation	• Validate file type using magic-byte inspection (not extension or MIME header alone); allowlist only jpeg/png/gif. • Re-encode images using a server-side library (e.g., Sharp) to strip any embedded payloads. • Serve uploaded files from an isolated domain or CDN without script execution context. • Disable XML external entity processing in the XML parser used by /b2b/v2/orders. • Apply Content-Security-Policy headers to prevent inline script execution.				

F06 GraphQL Introspection Enabled in Production

Risk Rating	Criticality	CVSS Score	Category	Affected Asset	Status
Medium	P3 – Medium	5.3	Security Misconfiguration	/graphql	Open
Description	The GraphQL endpoint has introspection enabled, allowing any unauthenticated client to query the full schema including all types, queries, mutations, and field names. This significantly accelerates attacker reconnaissance and reveals internal data models not exposed through the REST API.				
Evidence / Path	*POST /graphql body: {"query":"{__schema{types{name}}}"} → full schema returned* *Mutations including createUser, deleteUser, and updateUserRole visible in schema*				
Impact	Full API surface disclosure to unauthenticated attackers. Reveals hidden administrative mutations and sensitive field names that can be targeted in follow-on attacks.				
Recommendation	• Disable GraphQL introspection in production by setting introspection: false in the Apollo/graphql-js server config. • Implement query depth limiting and query complexity analysis to prevent DoS via deeply nested queries. • Require authentication for all GraphQL mutations. • Consider field-level authorisation using a library such as graphql-shield.				

F07 Prometheus Metrics & Application Configuration Exposed Publicly

Risk Rating	Criticality	CVSS Score	Category	Affected Asset	Status

Medium	P3 – Medium	5.3	Information Disclosure	/metrics, /rest/admin/application-configuration	Open
Description	The /metrics endpoint exposes Prometheus-format runtime telemetry (heap usage, request counts, GC stats) without authentication. The /rest/admin/application-configuration endpoint returns the application's full configuration object including mail server credentials and the JWT secret.				
Evidence / Path	*GET /metrics → # HELP process_heap_bytes ... nodejs_heap_size_used_bytes 45123456* *GET /rest/admin/application-configuration → JSON with smtp credentials, jwtSecret:* '[REDACTED_SECRET]				
Impact	Exposure of JWT signing secret (compounding Finding F02). SMTP credential leakage enabling email spoofing and spam. Operational intelligence (memory usage, endpoint hit rates) aids targeted DoS planning.				
Recommendation	• Restrict /metrics to internal networks or require bearer token authentication (standard Prometheus scrape config supports this). • Remove the /rest/admin/application-configuration endpoint or require admin-level authentication and strip secrets from the response. • Rotate all credentials disclosed by this endpoint immediately.				

F08 Missing Security HTTP Headers

Risk Rating	Criticality	CVSS Score	Category	Affected Asset	Status
Low	P4 – Low	3.1	Security Misconfiguration	All HTTP responses	Open
Description	HTTP responses lack several security headers that harden the application against client-side attacks. While Helmet.js is present as a dependency, it is not fully configured. Missing headers include Content-Security-Policy, Permissions-Policy, and a restrictive Referrer-Policy.				
Evidence / Path	*HTTP response headers: no Content-Security-Policy observed* *X-Frame-Options absent (clickjacking risk)* *Referrer-Policy not set (leaks URL data to third parties)*				
Impact	Increased risk of XSS exploitation (no CSP), clickjacking attacks (no X-Frame-Options), and sensitive URL leakage to analytics/ad networks.				
Recommendation	• Configure Helmet.js fully: enable contentSecurityPolicy, frameguard, referrerPolicy, and permittedCrossDomainPolicies. • Define a strict CSP that blocks inline scripts and restricts resource origins. • Add Permissions-Policy to disable unused browser features (camera, microphone, geolocation). • Automate header validation in CI using securityheaders.com API or equivalent.				

5. Remediation Roadmap

Prioritised by risk and effort

#	Finding	Risk	Effort	Priority Action
F01	SQL Injection	Critical	Medium	Parameterise all queries immediately
F02	JWT Weak Secret	Critical	Low	Rotate secret; pin RS256 — same-day fix
F03	IDOR / BOLA	High	Medium	Add server-side ownership checks
F04	Exposed /ftp/ & Keys	High	Low	Remove routes and move secrets to vault
F05	Unrestricted File Upload	High	Medium	Add magic-byte validation + re-encode images
F06	GraphQL Introspection	Medium	Low	Set introspection:false in server config
F07	Metrics / Config Exposed	Medium	Low	Restrict /metrics; remove secret from config API
F08	Missing Security Headers	Low	Low	Enable full Helmet.js config

6. Conclusion

The juice-shop.herokuapp.com application presents a broad and severe attack surface. Two critical-severity vulnerabilities (SQL injection and JWT forgery) are independently sufficient for a complete application compromise. The remaining high and medium findings compound this risk significantly.

Immediate remediation priority should be placed on F01 and F02, both of which require minimal engineering effort relative to their impact. All critical and high findings should be resolved before the application is used in any non-isolated environment.

A re-assessment is recommended after the critical and high findings have been remediated to validate fixes and identify any residual or newly introduced vulnerabilities.